



Sistemas de Tiempo Real

Práctica 1: Programación de microcontroladores. Interrupciones.

Ingeniería en Computación - 2026

Indicaciones Generales:

- Todas las experiencias deben realizarse utilizando el entorno de simulación **Proteus 8** y compiladas para la arquitectura AVR (**ATmega328P**) empleando C ANSI / Microchip Studio (o GCC-AVR).
- Para la evaluación de tiempos, utilice la herramienta de depuración/simulación de Proteus observando el tiempo real de ejecución (*Simulation Time*) en la barra de estado inferior.

Ejercicio 1: Control E/S Digital por Polling y Retardos

Abra el esquema basado en el microcontrolador **ATmega328P**. El circuito descrito en el esquema consiste de:

- **Entradas:** Dos pulsadores momentáneos conectados a los pines PD2 y PD3 (Puerto D), configurados con resistencias pull-up internas activadas.
- **Salidas:** Dos LEDs conectados a los pines PB0 y PB1 (Puerto B).

Desarrolle un firmware en C para este sistema tal que:

- Al iniciar la ejecución, ambos LEDs deben permanecer totalmente encendidos.
- Al detectar la presión de cualquiera de los dos pulsadores (vía la lectura directa de los registros de entrada PIND), los LEDs deben comenzar a parpadear de manera alternada con un período de **250 ms** por estado (250 ms encendido PB0/apagado PB1, y viceversa).
- Para el control temporal de la conmutación de los LEDs, utilice la función de retardo por software `_delay_ms()` de la biblioteca `<util/delay.h>`.
- d) *Verificación:* Observe la barra inferior del simulador Proteus para confirmar la precisión de la temporización de 250 ms.

Ejercicio 2: Temporización por Interrupciones y Hardware (Timer0)

Rehaga el **Ejercicio 1** conservando la misma funcionalidad del circuito y requerimientos del usuario, pero reestructurando la arquitectura del firmwaware. Utilice el módulo del TIMER0 con el prescaler para controlar el tiempo de espera y la interrupción para controlar el estado de los LEDs.

Ejercicio 3: Conversión Analógica-Digital (ADC) e Interfaz Multiplexada

Abra en Proteus 8 el esquema en el cual un **ATmega328P** adquiere una señal analógica y presenta su valor en una visualización multiplexada de 7 segmentos. El circuito descrito por el esquema tiene las siguientes características:

- **Entrada Analógica:** Un potenciómetro (simulando un transductor de temperatura o tensión) conectado al canal analógico ADC0 (PC0).
- **Interfaz de Salida:** Un bus de datos en el Puerto B (PB0--PB7) conectado a las entradas de dos Latches 74LS373 multiplexados, los cuales controlan cuatro displays de 7 segmentos hexadecimales. Las líneas de habilitación *Latch Enable* (LE) de los 74LS373 se controlan desde los pines PC1 y PC2.

Desarrolle un firmware en C para este sistema tal que:

- a) Configure el módulo ADC del ATmega328P:
 - Selección de canal ADC0.
 - Tensión de referencia interna o V_{CC} (5 V) con capacitor en *AREF*.
 - Prescaler del ADC configurado para mantener la frecuencia de muestreo dentro del rango recomendado (50 kHz – 200 kHz).
- b) Implemente la captura de datos mediante muestreo periódico por consulta de bandera (Polling en el registro ADCSRA).
- c) Presente el valor digital convertidor (10 bits, 0x000 – 0x3FF) en formato hexadecimal en los displays de 7 segmentos, generando los pulsos de control en los pines LE de los latches 74LS373 para multiplexar los dígitos.

Ejercicios Opcionales

A modo de profundización y experimentación sobre el Ejercicio 3:

1. **Conversión por Interrupción:** Modifique la adquisición del ADC para que funcione de manera asíncrona mediante la interrupción de conversión finalizada (ADC_vect).
2. **Visualización Decimal BCD:** Adapte la salida a los displays para mostrar el valor digital directamente en formato decimal (0 a 1023). Para ello, implemente un algoritmo de conversión de entero a dígitos BCD.
3. **Lectura de Voltaje Real:** Calcule la tensión de entrada equivalente (0,000 V a 5,000 V) y muestre el valor formateado con un dígito entero y tres decimales (ejemplo: 3.425).